

Grupo 10

Shallow Water con rompe olas usando PINNS

MA5307 – Análisis Numérico de EDP

Leonardo Ramos Osvaldo Cartagena

DIM, FCFM, Universidad de Chile

Junio 2026

Contexto

Objetivo

Ecuaciones Shallow Water 1D

Modelo propuesto

Versión 1

Versión 2

Resultados

Ejemplo estacionario

Ejemplo no-estacionario

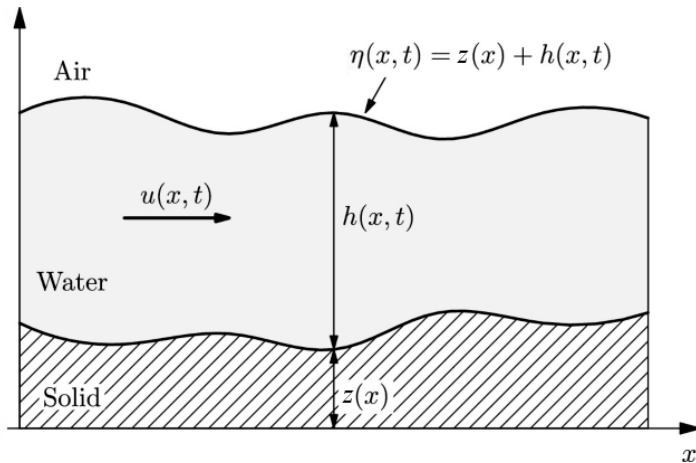
Rompeolas

Conclusiones

Bibliografía

Objetivo

Análisis comparativo de Shallow Water para topografía submarina con rompeolas usando PINNs vs (1)FVM.



Ecuaciones 1D de Shallow Water

$$\begin{cases} \partial_t h + \partial_x(hu) = 0 & \text{in } \Omega \times [0, T], \\ \partial_t(hu) + \partial_x(hu^2 + g\frac{h^2}{2}) = -gh\frac{dz}{dx} & \text{in } \Omega \times [0, T], \\ u(t=0) = u_0 & \text{in } \Omega \\ h(t=0) = h_0 & \text{in } \Omega \\ u|_{\partial\Omega} = 0 & \text{in } [0, T] \end{cases}$$

Donde u_0 , h_0 y z son funciones dadas, con u_0 compatible con la condición de borde espacial, es decir $u_0(0) = u_0(T) = 0$, $\Omega = (0, 1)$ y $T = 1$.

Ecuaciones Shallow Water 1D

Para soluciones suaves podemos definir las siguientes masa y energía, estas nos servirán para estudiar las propiedades conservativas del modelo.

$$M(t) = \int_{\Omega} h(x, t) dx$$

$$E(t) = \int_{\Omega} \left[\frac{1}{2} h u^2 + \frac{1}{2} g h^2 + g h z \right] (x, t) dx$$

Además, definiremos una superficie libre que representa la interfaz entre el agua y el aire, de forma que se cumpla lo siguiente:

$$\eta(x, t) = h(x, t) + z(x)$$

Physics Informed Neural Network

Una red neuronal $NN_{\theta}(x, t)$ es una función asociada a un conjunto de parametros θ . Las redes neuronales tienen buenas propiedades

1. Aproximacion Universal y otros
2. Diferenciabilidad

Por esto, son usadas para aproximar la solución de diversos problemas diferenciales. En general, se define una función $\mathcal{R}(\theta)$ no negativa, llamada corrientemente *loss*, que modeliza que tan 'mala' es la aproximación. Luego, se busca el minimizador de dicha loss mediante un algoritmo iterativo.

$$\theta^* = \arg \min_{\theta} \mathcal{R}(\theta)$$

En nuestro caso, buscamos definir una loss tal que cuanto mas pequeña sea, mas cerca estemos de la solucion de la EDP.

Physics Informed neural network. Version 1

Intentar aproximar (h, u) directamente por una red neuronal NN_θ , es decir proponer como solución $(h_\#, u_\#) = NN_\theta$. Luego, los residuos a minimizar son:

$$\mathcal{R}_{masa} = \|\partial_t h_\# + \partial_x(h_\# u_\#)\|_2^2$$

$$\mathcal{R}_{mom} = \|\partial_t(h_\# u_\#) + \partial_x(h_\# u_\#^2 + g \frac{h_\#^2}{2}) + gh \frac{dz}{dx}\|_2^2$$

$$\mathcal{R}_{ic} = \|u_\#(t=0) - u_0\|_2^2 + \|h_\#(t=0) - h_0\|_2^2$$

$$\mathcal{R}_{bc} = \|u(x=0)\|_2^2 + \|u(x=1)\|_2^2$$

$$\mathcal{R}_{total} = \mathcal{R}_{masa} + \mathcal{R}_{mom} + w_1 \mathcal{R}_{ic} + w_2 \mathcal{R}_{bc}$$

- Las normas anteriores se aproximan usando una discretización del dominio.
- Implementación numérica usando PyTorch en Google Collab.
- No funcionó

Resultados Versión 1

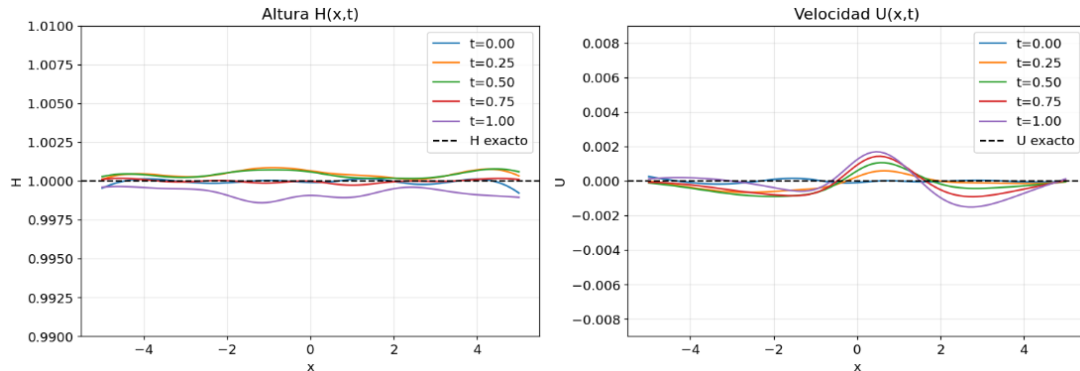


Figura: Ejemplo trivial se satisface

Resultados Versión 1

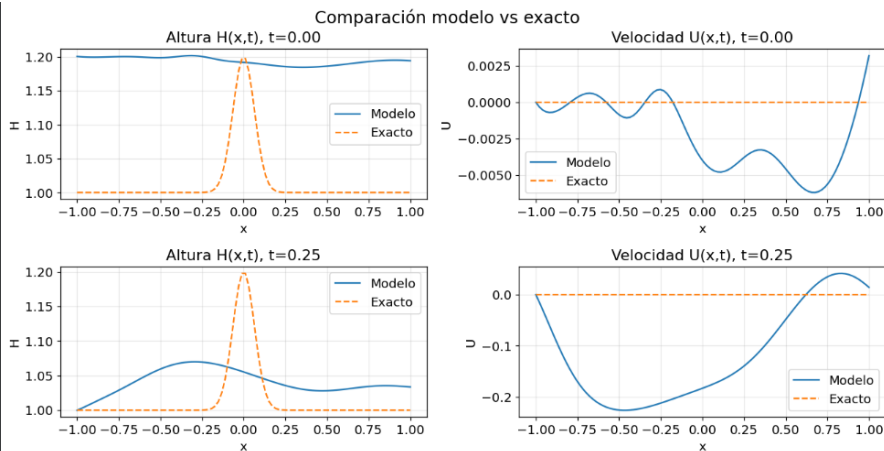


Figura: Ejemplo no trivial no converge

Physics Informed neural network. Version 2

En vez aproximar (h, u) directamente por una red neuronal NN_θ , usaremos como *ansatz* $(h_\#, u_\#) = (h_0, u_0) + tx(1 - x)NN_\theta$ (multiplicacion pointwise escondida). De esta manera, nuestro $(h_\#, u_\#)$ satisface la condicion inicial y la condicion de borde por **construcción**. Luego, los residuos a minimizar son solamente:

$$\mathcal{R}_{masa} = \|\partial_t h_\# + \partial_x(h_\# u_\#)\|_2^2$$

$$\mathcal{R}_{mom} = \|\partial_t(h_\# u_\#) + \partial_x(h_\# u_\#^2 + g \frac{h_\#^2}{2}) + gh_\# \frac{dz}{dx}\|_2^2$$

$$\mathcal{R}_{total} = \mathcal{R}_{masa} + \mathcal{R}_{mom}$$

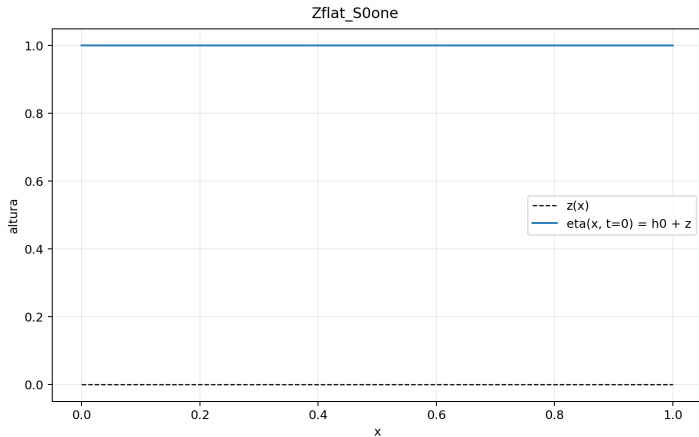
- Las normas anteriores se aproximan usando una discretización aleatoria del dominio.
- Implementación numérica usando PyTorch (Repo: SW-EDPN)

Resultados: Ejemplo estacionario

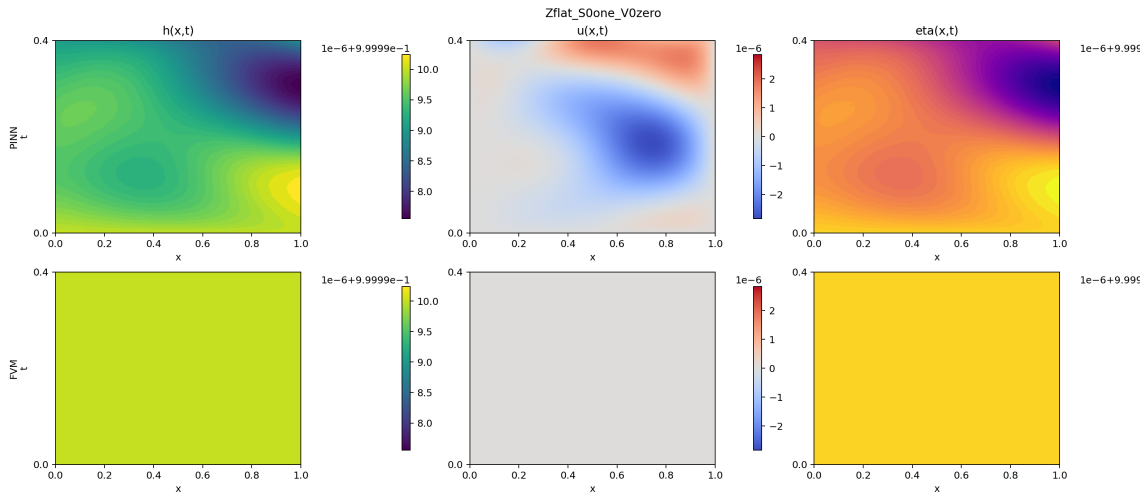
Ejemplo básico de validación.
El modelo tiene que ser capaz
de reproducir este ejemplo con
alta fidelidad, es especialmente
importante un error pequeño y
alto nivel de conservación.

Parámetros:

$$\begin{cases} \eta(x, 0) &= 1 \\ z(x) &= 0 \end{cases}$$

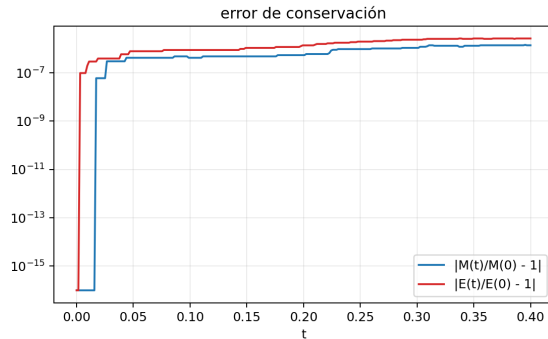
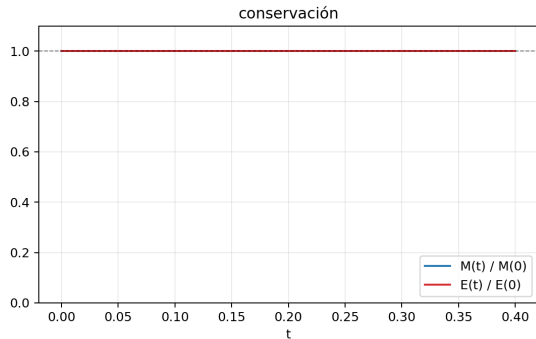


Resultados: Ejemplo estacionario



Resultados: Ejemplo estacionario

Zflat_S0one_V0zero

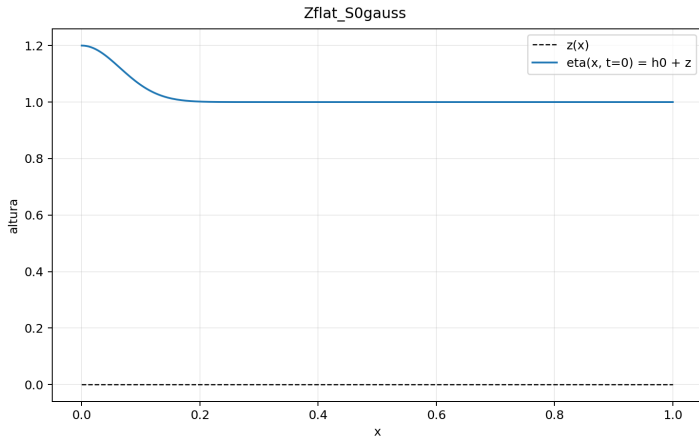


Resultados: Ejemplo plano no-estacionario

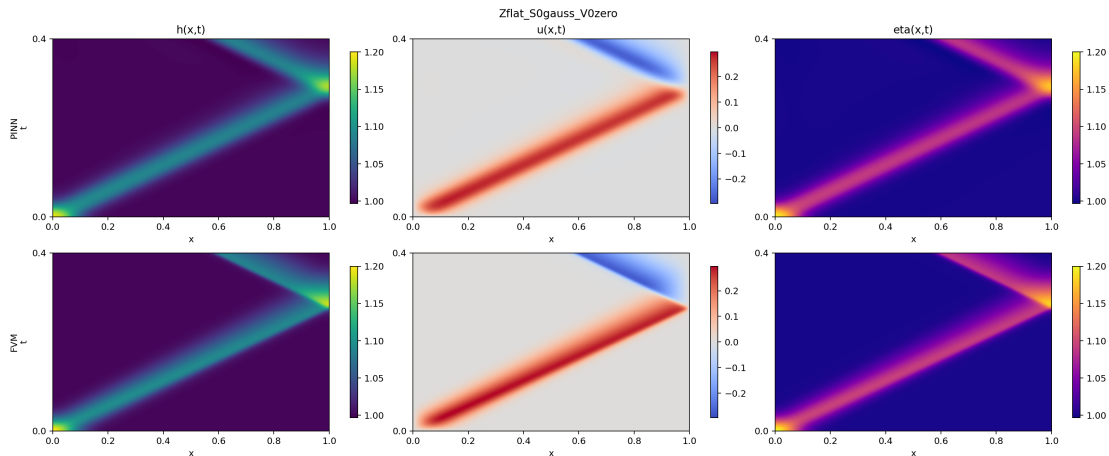
Ejemplo básico no-estacionario. El modelo tiene que simular la transformación de la energía potencial en energía cinética.

Parámetros:

$$\begin{cases} \eta(x, 0) = 1,0 + 0,2 \exp(-120x^2) \\ z(x) = 0 \end{cases}$$

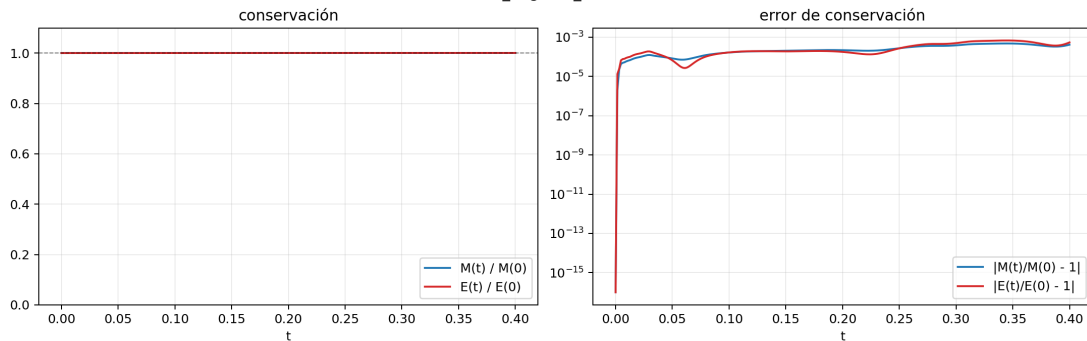


Resultados: Ejemplo no-estacionario



Resultados: Ejemplo no-estacionario

Zflat_S0gauss_V0zero

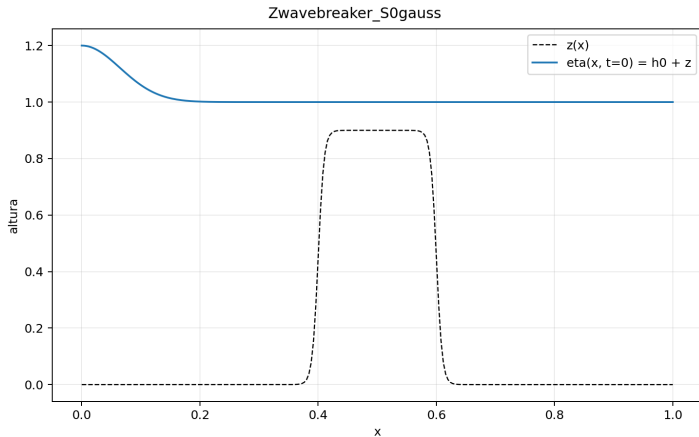


Resultados: Rompeolas

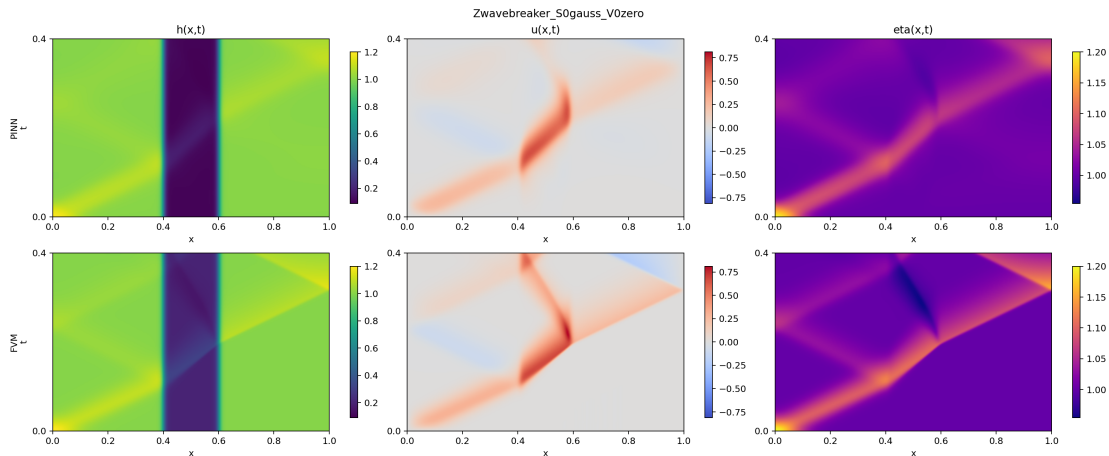
Rompeolas en setting no-estacionario. El modelo tiene que respetar los cambios de altura y velocidad en y lejos del rompeolas, es especialmente importante que es el rompeolas logre disminuir la velocidad de la ola.

Parámetros:

$$\begin{cases} \eta(x, 0) = 1,0 + 0,2 \exp(-120x^2) \\ z(x) = 0,45[\tanh(100(x - 0,4)) - \tanh(100(x - 0,6))] \end{cases}$$

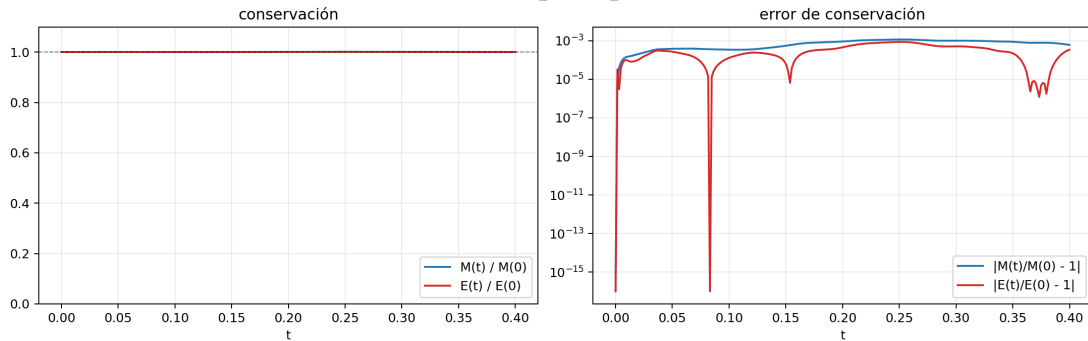


Resultados: Rompeolas



Resultados: Rompeolas

Zwavebreaker_S0gauss_V0zero



Conclusiones

El modelo PINN fue consistente con la intuición física, mostrando buenas propiedades en sus dimensiones cuantitativas (error, conservación, etc) como cualitativas (efecto del rompeolas).

La dinámica es consistente, en términos generales, con la referencia de FVM; presentando mejoras comparativas en el efecto rompeolas (disminución de la velocidad), pero mostrando inconsistencias en interacciones complejas (rebote del agua).

Bibliografía

- [1] Christophe Berthon and Christophe Chalons. A fully well-balanced, positive and entropy-satisfying Godunov-type method for the shallow-water equations. *Mathematics of Computation*, 85(299):1281–1307, May 2016. Article electronically published on September 15, 2015. doi:10.1090/mcom3045.
- [2] I.E. Lagaris, A. Likas, and D.I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998. doi:10.1109/72.712178.
- [3] Nathan Doumèche, Gérard Biau, and Claire Boyer. On the convergence of PINNs. arXiv preprint arXiv:2305.01240, 2026. URL: <https://arxiv.org/abs/2305.01240>.
- [4] Siddhartha Mishra. Estimates on the generalization error of physics-informed neural networks (PINNs) for approximating PDEs. *IMA Journal of Numerical Analysis*, 43(3):1591–1636, 2023. doi:10.1093/imanum/drac084.
- [5] Toro, Eleuterio F. The Shallow Water Equations, Computational Algorithms for Shallow Water Equations, 2024 *Springer Nature Switzerland* isbn=978-3-031-61395-1, doi=10.1007/978-3-031-61395-1_1, url=https://doi.org/10.1007/978-3-031-61395-1_1,

Gracias!